



Department of
Computer Science and Engineering
University of Dhaka

Title: Implementation of Client-Server
Communication using Socket Programming

CSE 3111: Computer Networking Lab

Batch: 29/3rd Year 1st Semester 2024

prepared by: Mehedi Hasan(22) and Biplob Pal(50)

Contents

1	Introduction	2
2	Objectives	2
3	Design Details	2
4	Implementation	3
4.1	Server.java Code	3
4.2	Client.java Code	5
5	Result Analysis	6
6	Discussion	7

1 Introduction

Socket programming is a fundamental technique in computer networking that enables communication between two processes over a network. It provides a standardized interface for applications to send and receive data using protocols like TCP or UDP. In Java, socket programming involves creating sockets—endpoints for communication—that can be connection-oriented (using TCP for reliable, ordered delivery) or connection-less (using UDP for faster but less reliable transmission). Sockets abstract the underlying network details, allowing developers to focus on application logic while handling data transmission via input and output streams.

Multi-threaded socket programming extends basic socket programming by incorporating threads to manage concurrent connections. In a basic socket setup, a server can typically handle only one client at a time, leading to blocking behavior where subsequent clients must wait. Multi-threading addresses this by assigning each client connection to a separate thread, enabling the server to process multiple requests simultaneously without interrupting ongoing communications. This is achieved by extending the `Thread` class or implementing `Runnable`, where each thread manages its own input/output streams for independent client-server interactions.

The necessity of socket programming and multi-threading for designing a chat application is evident in real-time communication scenarios. A chat application requires reliable, bidirectional data exchange between clients and a server. Basic sockets provide the foundation for establishing connections and transferring messages, while multi-threading ensures scalability—allowing multiple users to chat concurrently without performance degradation. Without multi-threading, the application would be limited to single-user interactions, making it unsuitable for group or multi-user chats. This combination enables features like instant messaging, where the server can relay or respond to messages from various clients in parallel, mimicking real-world applications like WhatsApp or IRC servers.

2 Objectives

- To implement a multi-threaded server that can handle connections from multiple clients simultaneously, allowing concurrent chat sessions.
- To develop a fully functional chat application where clients can send single or multiple sentences to the server, and the server can reply to each client independently.
- To incorporate termination conditions for both client and server sides to gracefully end communication sessions.

3 Design Details

The chat application is designed using Java's socket programming with multi-threading to support multiple clients. The system consists of a server that listens for incoming connections and spawns a new thread for each client, ensuring independent communication channels. Clients connect to the server, send messages, and receive replies. The termination condition is triggered when a client sends the message "Exit", which closes the connection for that client. The server continues running to accept new connections.

Step-by-step implementation process (textual representation; a flowchart could be added here for visualization):

1. **Server Initialization:** Created a `ServerSocket` on a specified port 5000 to listen for client connections in an infinite loop.
2. **Accept Client Connection:** Upon receiving a connection request, accepted it to create a plain `Socket` for communication.
3. **Create Input/Output Streams:** Extract `DataInputStream` (for reading) and `DataOutputStream` (for writing) from the socket.

4. **Spawn Client Handler Thread:** Instantiate a ClientHandler object (extending Thread), passing the socket and streams as parameters, and start the thread.
5. **Client Handler Logic:** In the thread's run() method, loop to read messages from the client using readUTF(). If the message is "Exit", close the connection. Otherwise, print the message to the server console, prompt for a reply via System.in, and send the reply using writeUTF().
6. **Client Side:** Create a Socket connecting to the server's IP and port. Set up DataInputStream and DataOutputStream. In a loop, read user input from console, send it to the server using writeUTF(), and read the server's reply using readUTF(). Exit if "Exit" is sent.
7. **Error Handling:** Use try-catch blocks for IOExceptions to handle connection issues gracefully.
8. **Termination:** Client terminates on "Exit". Server runs indefinitely but closes individual client sockets upon termination.

This design overcomes the limitations of basic socket programming by using threads to prevent blocking, allowing the server to handle multiple clients without queuing.

4 Implementation

The implementation is done in Java, with two main files: Server.java (containing the Server class and ClientHandler inner class) and Client.java.

4.1 Server.java Code

```

1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4
5 public class server {
6     private static Map<String, String> accountPin = Collections.synchronizedMap(new HashMap
7     <>());
8     private static Map<String, Integer> accountBalance = Collections.synchronizedMap(new
9     HashMap<>());
10
11     public static void main(String[] args) {
12         loadDatabase();
13         try {
14             ServerSocket server = new ServerSocket(7777);
15             System.out.println("Server started on port 7777");
16             while (true) {
17                 Socket socket = server.accept();
18                 System.out.println("New client connected: " + socket.getInetAddress());
19                 new ClientHandler(socket).start();
20             }
21         } catch (Exception e) {
22             e.printStackTrace();
23         }
24     }
25
26     private static void loadDatabase() {
27         try (BufferedReader br = new BufferedReader(new FileReader("database.txt"))) {
28             String line;
29             while ((line = br.readLine()) != null) {
30                 String[] parts = line.split(",");
31                 accountPin.put(parts[0], parts[1]);
32                 accountBalance.put(parts[0], Integer.parseInt(parts[2]));
33             }
34             System.out.println("Database loaded successfully.");
35         } catch (Exception e) {
36             System.out.println("Error loading database: " + e.getMessage());
37         }
38     }
39 }

```

```

36     }
37
38     private static synchronized void saveDatabase() {
39         try (BufferedWriter bw = new BufferedWriter(new FileWriter("database.txt"))) {
40             for (String card : accountBalance.keySet()) {
41                 bw.write(card + "," + accountPin.get(card) + "," + accountBalance.get(card))
42             ;
43                 bw.newLine();
44             }
45         } catch (Exception e) {
46             System.out.println("Error saving database: " + e.getMessage());
47         }
48     }
49
50     static class ClientHandler extends Thread {
51         Socket socket;
52         DataInputStream inputDS;
53         DataOutputStream outputDS;
54         String authenticatedCard = null;
55
56         public ClientHandler(Socket socket) {
57             this.socket = socket;
58             try {
59                 inputDS = new DataInputStream(socket.getInputStream());
60                 outputDS = new DataOutputStream(socket.getOutputStream());
61             } catch (Exception e) {
62                 e.printStackTrace();
63             }
64         }
65
66         public void run() {
67             try {
68                 while (true) {
69                     String message = inputDS.readUTF();
70
71                     if (message.startsWith("AUTH:")) {
72                         String[] parts = message.split(":");
73                         if (parts.length == 3) {
74                             String card = parts[1];
75                             String pin = parts[2];
76                             if (accountPin.containsKey(card) && accountPin.get(card).equals(
77                                 pin)) {
78                                 authenticatedCard = card;
79                                 outputDS.writeUTF("AUTH_OK");
80                                 System.out.println("Client authenticated: " + card);
81                             } else {
82                                 outputDS.writeUTF("AUTH_FAIL");
83                                 System.out.println("Failed authentication attempt for card:
84                                     " + card);
85                             }
86                         } else {
87                             outputDS.writeUTF("INVALID_FORMAT");
88                             System.out.println("Invalid AUTH format received");
89                         }
90                     } else if (message.equals("BALANCE_REQ")) {
91                         if (authenticatedCard != null) {
92                             int balance = accountBalance.get(authenticatedCard);
93                             outputDS.writeUTF("BALANCE_RES:" + balance);
94                             System.out.println("Balance request for " + authenticatedCard +
95                                 ": " + balance);
96                         } else {
97                             outputDS.writeUTF("AUTH_REQUIRED");
98                             System.out.println("Balance request without authentication");
99                         }
100                     } else if (message.startsWith("WITHDRAW:")) {
101                         if (authenticatedCard != null) {

```

```

100         int amount = Integer.parseInt(message.split(":")[1]);
101         int balance = accountBalance.get(authenticatedCard);
102         if (balance >= amount) {
103             accountBalance.put(authenticatedCard, balance - amount);
104             saveDatabase();
105             outputDS.writeUTF("WITHDRAW_OK");
106             System.out.println("Withdrawal of " + amount + " for " +
authenticatedCard + ". New balance: " + (balance - amount));
107         } else {
108             outputDS.writeUTF("INSUFFICIENT_FUNDS");
109             System.out.println("Failed withdrawal (insufficient funds)
for " + authenticatedCard + ": " + amount);
110         }
111         } else {
112             outputDS.writeUTF("AUTH_REQUIRED");
113             System.out.println("Withdrawal request without authentication");
114         }
115     } else if (message.equals("EXIT")) {
116         outputDS.writeUTF("EXIT");
117         System.out.println("Client disconnected: " + socket.getInetAddress()
);
118     };
119     socket.close();
120     break;
121 } else {
122     outputDS.writeUTF("UNKNOWN_COMMAND");
123     System.out.println("Unknown command received: " + message);
124 }
125 }
126 } catch (Exception e) {
127     System.out.println("Connection lost with client: " + socket.getInetAddress()
);
128 }
129 }
130 }
131 }
132 }

```

4.2 Client.java Code

```

1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4
5 class MessageReceiver extends Thread {
6     DataInputStream inputDS;
7
8     public MessageReceiver(DataInputStream inputDS) {
9         this.inputDS = inputDS;
10    }
11
12    public void run() {
13        try {
14            String message;
15            while (true) {
16                message = inputDS.readUTF();
17                System.out.println("\nServer: " + message);
18                if (message.equals("EXIT"))
19                    break;
20                System.out.print("You: ");
21            }
22        } catch (Exception e) {
23        }
24    }
25 }
26
27 public class Roll_22_Client {

```

```

28 public static void main(String[] args) {
29     try {
30         Scanner sc = new Scanner(System.in);
31         Socket socket = new Socket("10.170.251.21", 7777);
32         DataInputStream inputDS = new DataInputStream(socket.getInputStream());
33         DataOutputStream outputDS = new DataOutputStream(socket.getOutputStream());
34         MessageReceiver receiver = new MessageReceiver(inputDS);
35         receiver.start();
36         System.out.print("Enter card number: ");
37         String card = sc.nextLine();
38         System.out.print("Enter PIN: ");
39         String pin = sc.nextLine();
40         outputDS.writeUTF("AUTH:" + card + ":" + pin);
41         outputDS.flush();
42         while (true) {
43             System.out.print("You: ");
44             String text = sc.nextLine();
45             if (text.equalsIgnoreCase("EXIT")) {
46                 outputDS.writeUTF("EXIT");
47                 outputDS.flush();
48                 break;
49             }
50             if (text.equalsIgnoreCase("BALANCE_REQ")) {
51                 outputDS.writeUTF("BALANCE_REQ");
52             } else if (text.toUpperCase().startsWith("WITHDRAW")) {
53                 String[] parts = text.split(" ");
54                 if (parts.length != 2) {
55                     System.out.println("Usage: WITHDRAW <amount>");
56                     continue;
57                 }
58                 outputDS.writeUTF("WITHDRAW:" + parts[1]);
59             } else {
60                 System.out.println("Ue BALANCE_REQ, WITHDRAW <amount> EXIT");
61             }
62             outputDS.flush();
63         }
64         receiver.join();
65         socket.close();
66         sc.close();
67     } catch (Exception e) {
68     }
69 }
70 }

```

Screenshots:

```

mehedi@Mehedi-ThinkPad-L480:~/CSE 3-1/CSE-3101-Networking/Networking Lab-3$ java Roll_50_Server
Database loaded successfully.
Server started on port 7777
New client connected: /127.0.0.1
Client authenticated: 1234
Withdrawal of 300 for 1234. New balance: 700
Balance request for 1234: 700
Client disconnected: /127.0.0.1
mehedi@Mehedi-ThinkPad-L480:~/CSE 3-1/CSE-3101-Networking/Networking Lab-3$

```

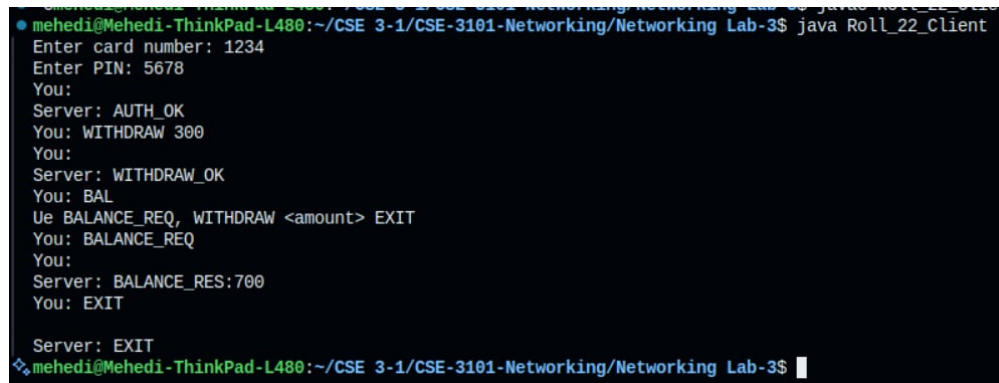
Figure 1: Server Side Console

5 Result Analysis

The application was tested with multiple clients connecting to the server simultaneously.

Example Output (Server Side):

Server started on port 5000



```

mehedi@mehedi-ThinkPad-L480:~/CSE 3-1/CSE-3101-Networking/Networking Lab-3$ java Roll_22_Client
Enter card number: 1234
Enter PIN: 5678
You:
Server: AUTH_OK
You: WITHDRAW 300
You:
Server: WITHDRAW_OK
You: BAL
Ue BALANCE_REQ, WITHDRAW <amount> EXIT
You: BALANCE_REQ
You:
Server: BALANCE_RES:700
You: EXIT
Server: EXIT
mehedi@mehedi-ThinkPad-L480:~/CSE 3-1/CSE-3101-Networking/Networking Lab-3$

```

Figure 2: Client Side Console

```

New client connected: Socket[addr=/127.0.0.1,port=54321,localport=5000]
New client connected: Socket[addr=/127.0.0.1,port=54322,localport=5000]
Client Socket[addr=/127.0.0.1,port=54321,localport=5000]: Hello from Client 1
Server reply to Socket[addr=/127.0.0.1,port=54321,localport=5000]: Hi Client 1
Client Socket[addr=/127.0.0.1,port=54322,localport=5000]: Hi from Client 2
Server reply to Socket[addr=/127.0.0.1,port=54322,localport=5000]: Hello Client 2
Client Socket[addr=/127.0.0.1,port=54321,localport=5000]: Exit
Client Socket[addr=/127.0.0.1,port=54321,localport=5000] exited.

```

Example Output (Client 1 Side):

```

Connected to server. Type 'Exit' to quit.
Client message: Hello from Client 1
Server: Hi Client 1
Client message: Exit

```

Description: The server handles multiple clients concurrently, printing messages from each and allowing independent replies. Termination occurs smoothly on "Exit". If multiple messages are sent quickly, threads ensure they are processed without loss, though console input for replies requires careful sequencing by the server operator.

(Include screenshot images of console outputs here using)

6 Discussion

Basic socket programming allows simple client-server communication but suffers from significant drawbacks, such as inability to handle multiple clients concurrently. In a non-threaded setup, the server processes one client at a time, causing others to block or fail to connect, leading to poor scalability and responsiveness. Additionally, if a client sends multiple messages without waiting for replies, the server may only receive the first one due to buffering issues, resulting in message loss.

Multi-threaded socket programming overcomes these by dedicating a thread per client, enabling parallel processing. This ensures the server remains available for new connections while handling ongoing chats, preventing blocking and message queuing problems. For instance, in our implementation, each ClientHandler thread manages its own streams, allowing independent communication without interference.

Through this lab, we learned the practical application of threads in networking, including synchronization challenges (e.g., shared console input) and the importance of exception handling for robust connections. Difficulties faced included managing interleaved console inputs on the server side, which could lead to mismatched replies in high-traffic scenarios—potentially resolvable with a GUI or queue-based input system. Overall, this enhanced our understanding of concurrent programming for real-time applications.